Lab Notebook: EZIE-Mag vs Phone Accelerometer Accuracy

For this lab, we will compare the accuracy of the phone's accelerometer (data taken with Phyphox) with the fancy NASA-grade accelerometer. We will pull both taped down to a cart through the rec center (on the cart to help eliminate noise) and integrate from the acceleration data to find displacement (the length the cart was pulled); then compare the calculated positions to each other and to our manually measured displacement.

Manually measured length

We used a tape measure to determine that the length the cart was pulled (from the edge of the first basketball court to the opposite edge of the third basketball court) was $170ft9in \pm 1in$. The in mark was very close to precise, but we will allow some error from the tape measure perhaps not being fully straight (but not too much because we got it pretty straight). We need to convert this to meters.

In [1]:
```
l_m_in_feet = 170 + (9/12)
l_m_in_feet_err = 1/12
#conversion : 1ft = 0.3048m
l_m = l_m_in_feet * 0.3048
l_m_err = l_m_in_feet_err * 0.3048
print(l_m, "+/-", l_m_err, "m")
```

```
52.0446 +/- 0.0254 m
```

The length we will compare our calculated position values to is:

$$d_{measured} = 52.044 \pm 0.025m$$

In [2]:
```
import pandas as pd
P_df = pd.read_csv("Data/AccelerometerPhone.csv")
P_df # phone acceleration data
```

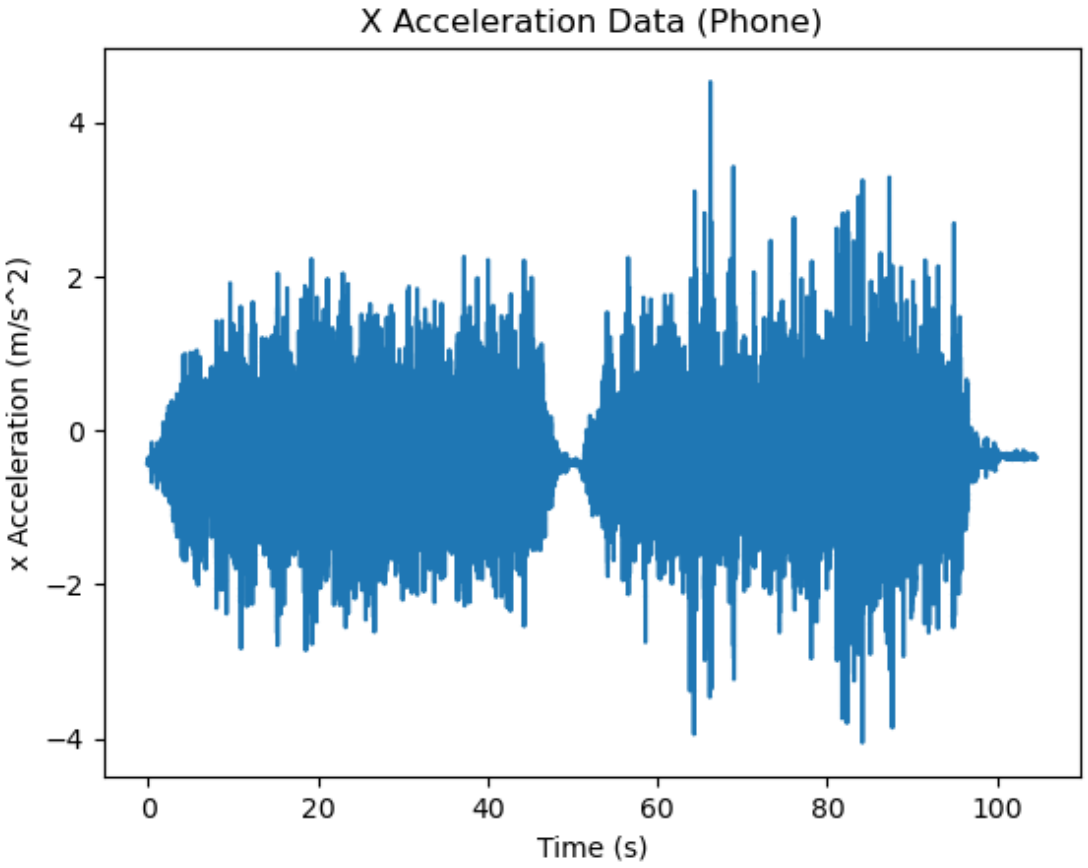
0	0.016224	-0.421137	0.265005	9.479167
1	0.018338	-0.425922	0.275175	9.536594
2	0.020462	-0.425922	0.270389	9.584451
3	0.022598	-0.421137	0.260818	9.560523
4	0.024743	-0.425922	0.269791	9.445667
...	...	...	...	...
48459	104.527239	-0.340977	0.265005	9.471989
48460	104.529399	-0.345763	0.255434	9.448658
48461	104.531549	-0.340977	0.259621	9.391231
48462	104.533702	-0.336192	0.264407	9.343374
48463	104.535865	-0.355334	0.264407	9.348160

48464 rows × 4 columns

I'm going to graph it to see where the data appears to start, which trial is which, figure out how to cut out noise, etc...

In [3]:
```
import numpy as np
#error due to reported phyphox accuracy limitations: 0.005 m/s^2
P_df["Acceleration Error (m/s^2)"] = 0.005
P_df["Acceleration z corrected for g (m/s^2)"] = (P_df["Acceleration z (m/s
                                                  - np.mean(P_df["Accelerat

P_a_abs = np.sqrt( P_df["Acceleration x (m/s^2)"]**2
                 + P_df["Acceleration y (m/s^2)"]**2
                 + P_df["Acceleration z corrected for g (m/s^2)"]**2)
P_df["Absolute acceleration (m/s^2)"] = P_a_abs
P_t = P_df["Time (s)"]
P_aerr = P_df["Acceleration Error (m/s^2)"]

import matplotlib.pyplot as plt
plt.errorbar(P_t, P_df["Acceleration x (m/s^2)"], yerr=P_aerr, fmt="-")
plt.xlabel("Time (s)")
plt.ylabel("x Acceleration (m/s^2)")
plt.title("X Acceleration Data (Phone)")
```

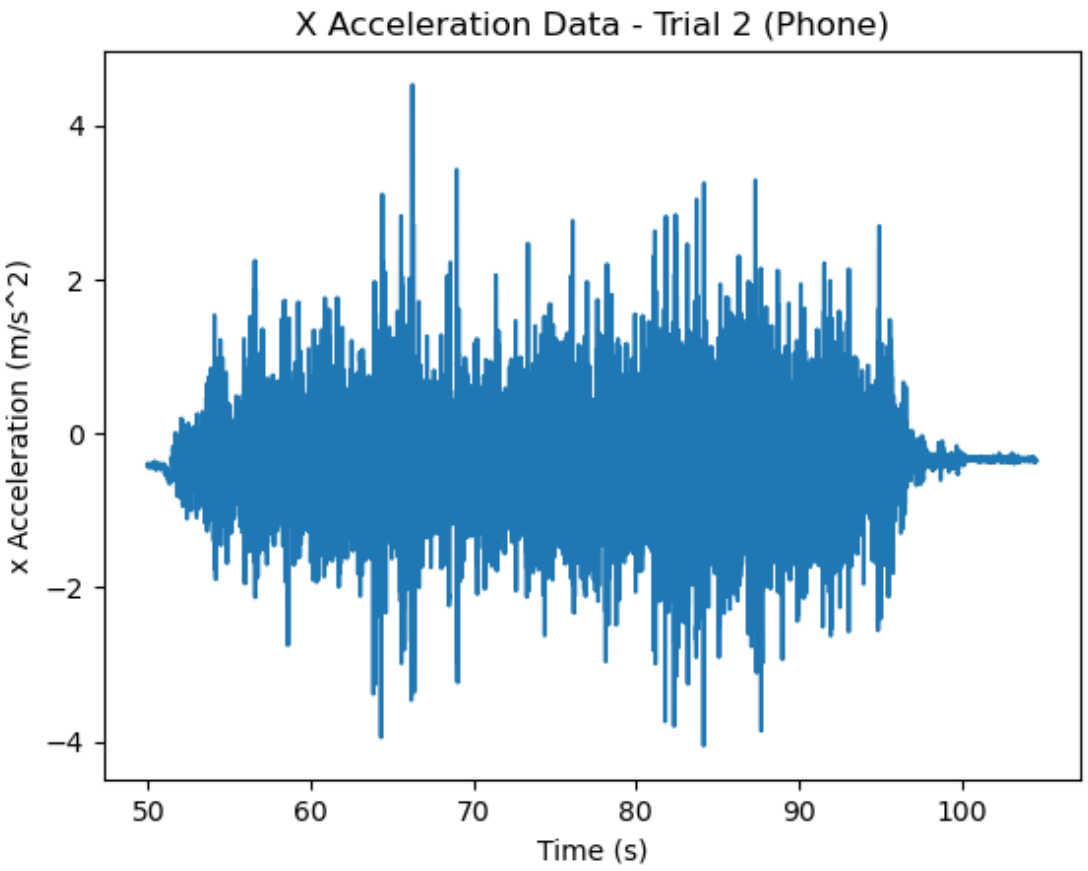
Out[3]:
```
Text(0.5, 1.0, 'X Acceleration Data (Phone)')
```



The data from the second trial will be more valid because the first trial was less precise in terms of length. I'm going to isolate the second trial and then try to take out noise.

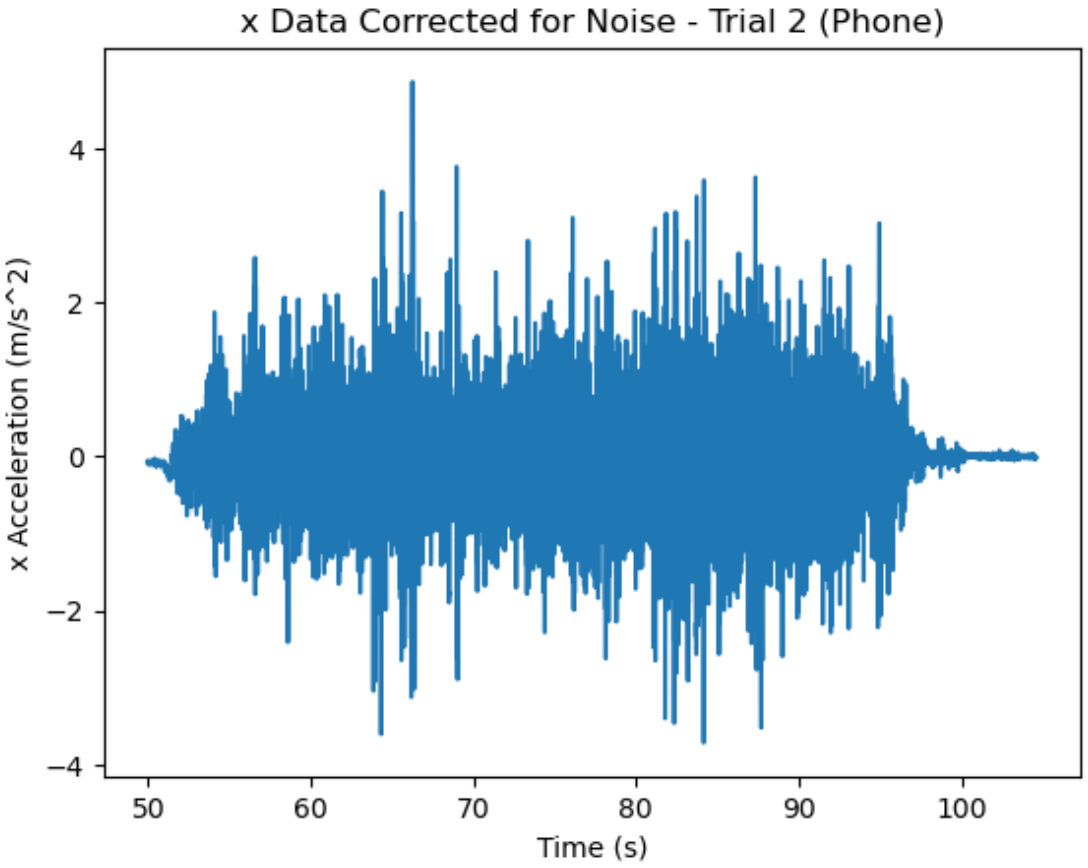
In [4]:
```
#isolating second trial

P_trial2 = P_df[P_df["Time (s)"] > 50]
P_t_trial2 = P_trial2["Time (s)"]
P_ax_trial2 = P_trial2["Acceleration x (m/s^2)"]
P_aerr_trial2 = P_trial2["Acceleration Error (m/s^2)"]

plt.errorbar(P_t_trial2, P_ax_trial2, yerr=P_aerr_trial2, fmt="-")
plt.xlabel("Time (s)")
plt.ylabel("x Acceleration (m/s^2)")
plt.title("X Acceleration Data - Trial 2 (Phone)")
```

Out[4]:
```
Text(0.5, 1.0, 'X Acceleration Data - Trial 2 (Phone)')
```



In [5]:
```
#removing noise

Pdf_noise = P_df[P_df["Time (s)"] > 100]
P_ax_noise = Pdf_noise["Acceleration x (m/s^2)"]

axnoise = np.mean(P_ax_noise)
P_ax_corr = P_ax_trial2 - axnoise


plt.errorbar(P_t_trial2, P_ax_corr, yerr=P_aerr_trial2, fmt="-")
plt.xlabel("Time (s)")
plt.ylabel("x Acceleration (m/s^2)")
plt.title("x Data Corrected for Noise - Trial 2 (Phone)")
```

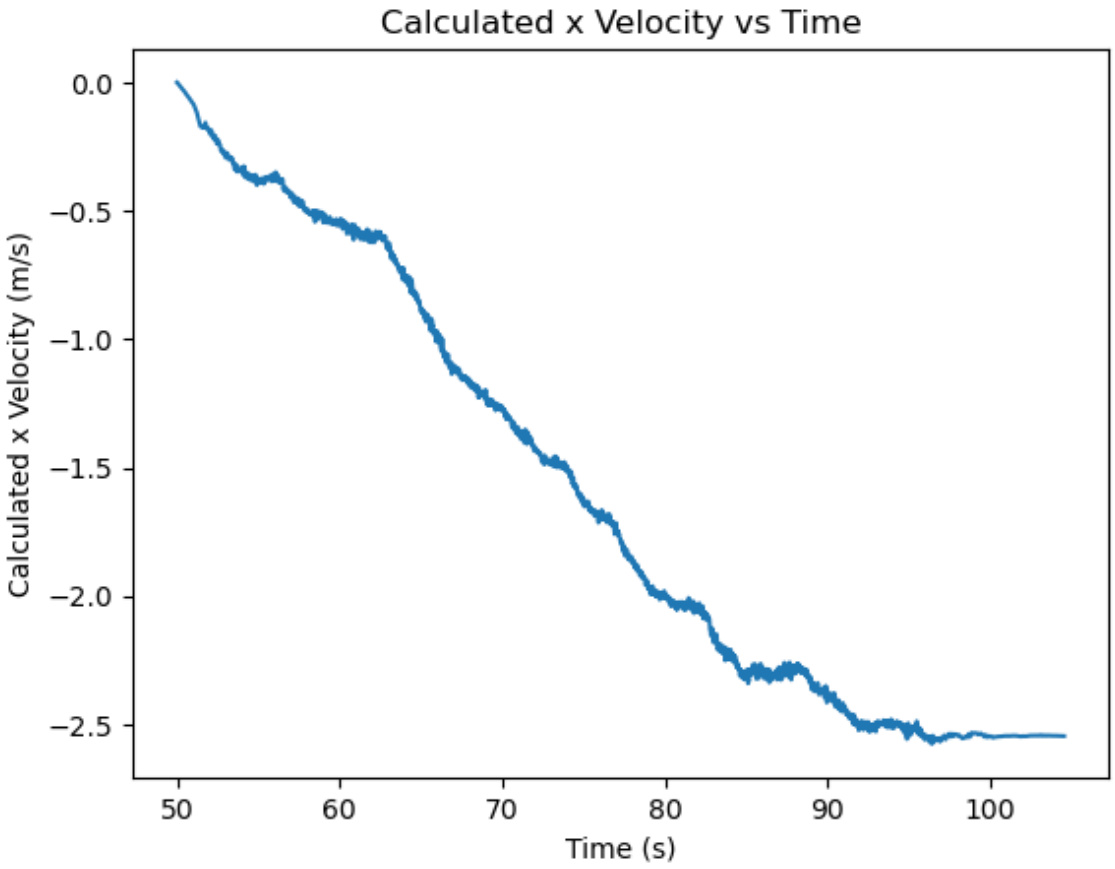
Out[5]:
```
Text(0.5, 1.0, 'x Data Corrected for Noise - Trial 2 (Phone)')
```



Now I need to integrate twice to find the displacement from the velocity.

In [6]:
```
#I love copy and paste
#FOR X ACCELERATION

xvelocity=[]
for i in range(P_t_trial2.size):
    xvelocity.append(np.trapz(P_ax_corr [0:i], P_t_trial2[0:i]))

plt.plot(P_t_trial2, xvelocity)
plt.xlabel("Time (s)")
plt.ylabel("Calculated x Velocity (m/s)")
plt.title("Calculated x Velocity vs Time")
```

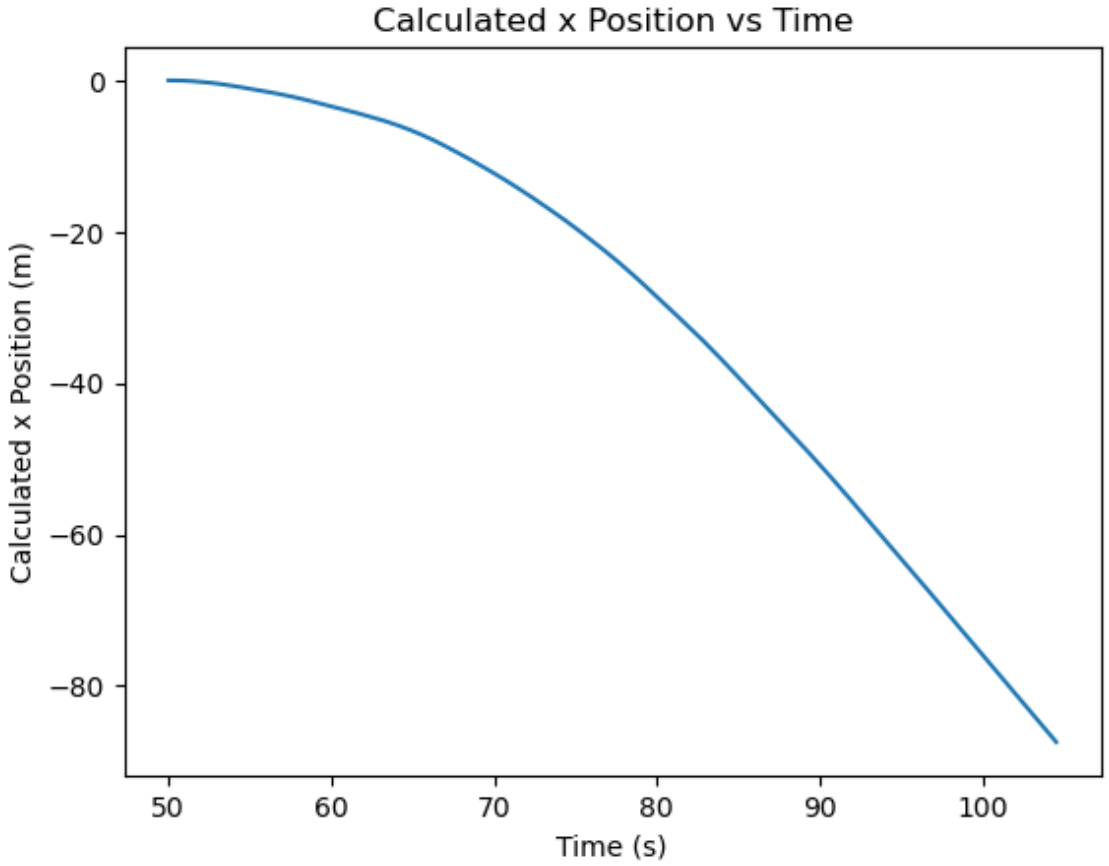
Out[6]: Text(0.5, 1.0, 'Calculated x Velocity vs Time')



2.5 m/s over 50 s drift

In [7]:
```
xposition=[]
for i in range(P_t_trial2.size):
    xposition.append(np.trapz(xvelocity [0:i], P_t_trial2[0:i]))

plt.plot(P_t_trial2, xposition)
plt.xlabel("Time (s)")
plt.ylabel("Calculated x Position (m)")
plt.title("Calculated x Position vs Time")
```

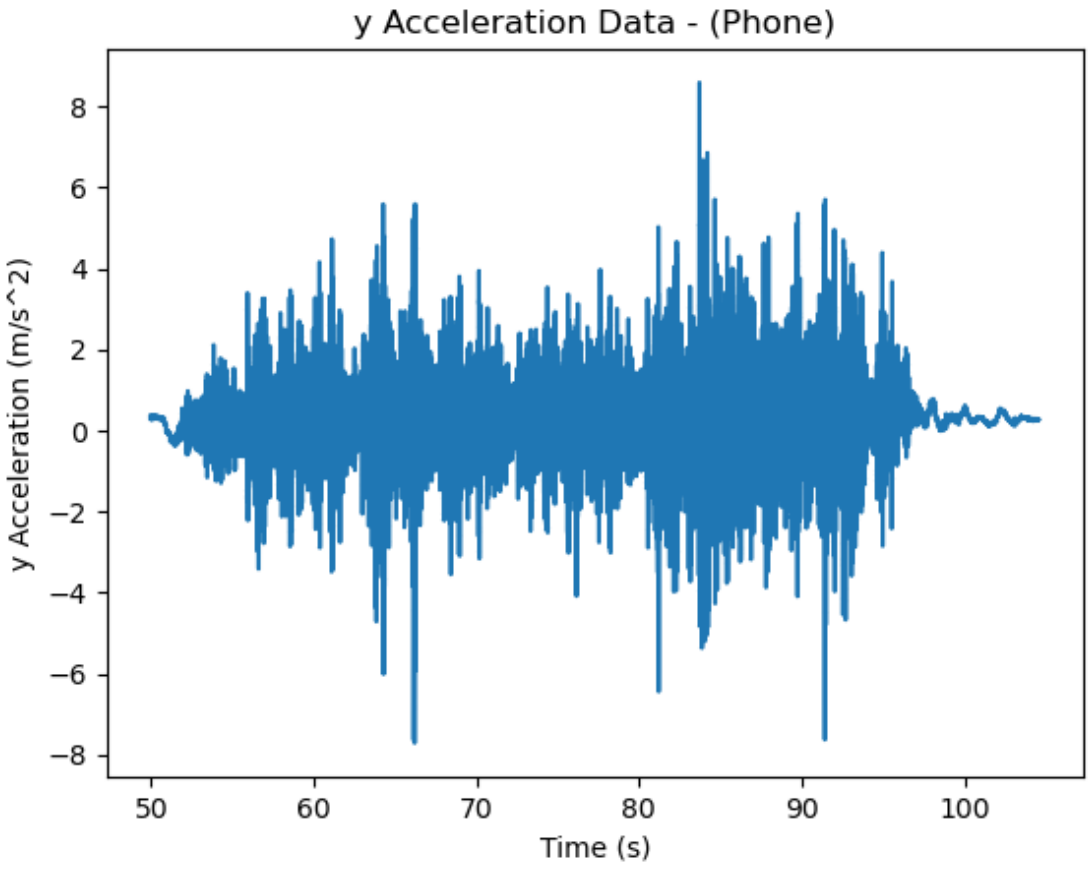
Out[7]:
```
Text(0.5, 1.0, 'Calculated x Position vs Time')
```



The velocity shows a general drift of about 0.05 m/s^2 - over 50 seconds, this can propogate to a more significant impact?

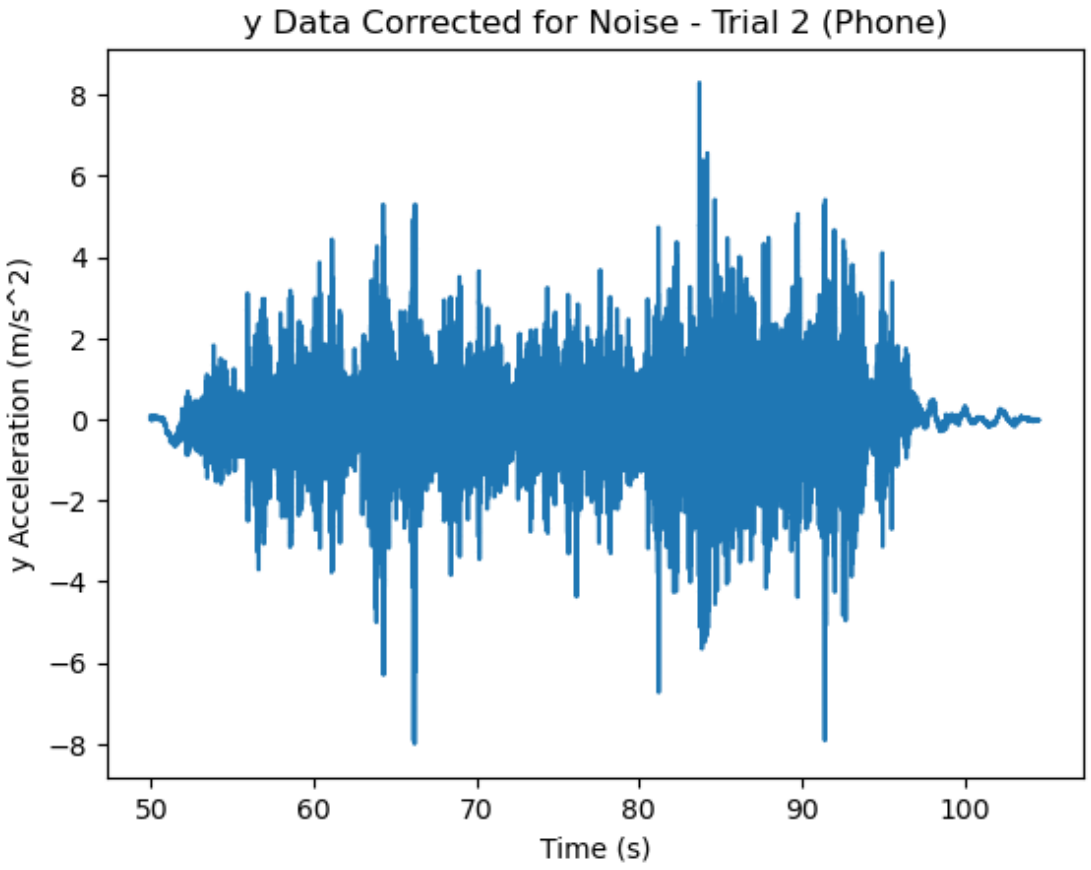
In [57]:
```
P_ay_trial2 = P_trial2["Acceleration y (m/s^2)"]

plt.errorbar(P_t_trial2, P_ay_trial2, yerr=P_aerr_trial2, fmt="-")
plt.xlabel("Time (s)")
plt.ylabel("y Acceleration (m/s^2)")
plt.title("y Acceleration Data - (Phone)")
```

Out[57]:
```
Text(0.5, 1.0, 'y Acceleration Data - (Phone)')
```



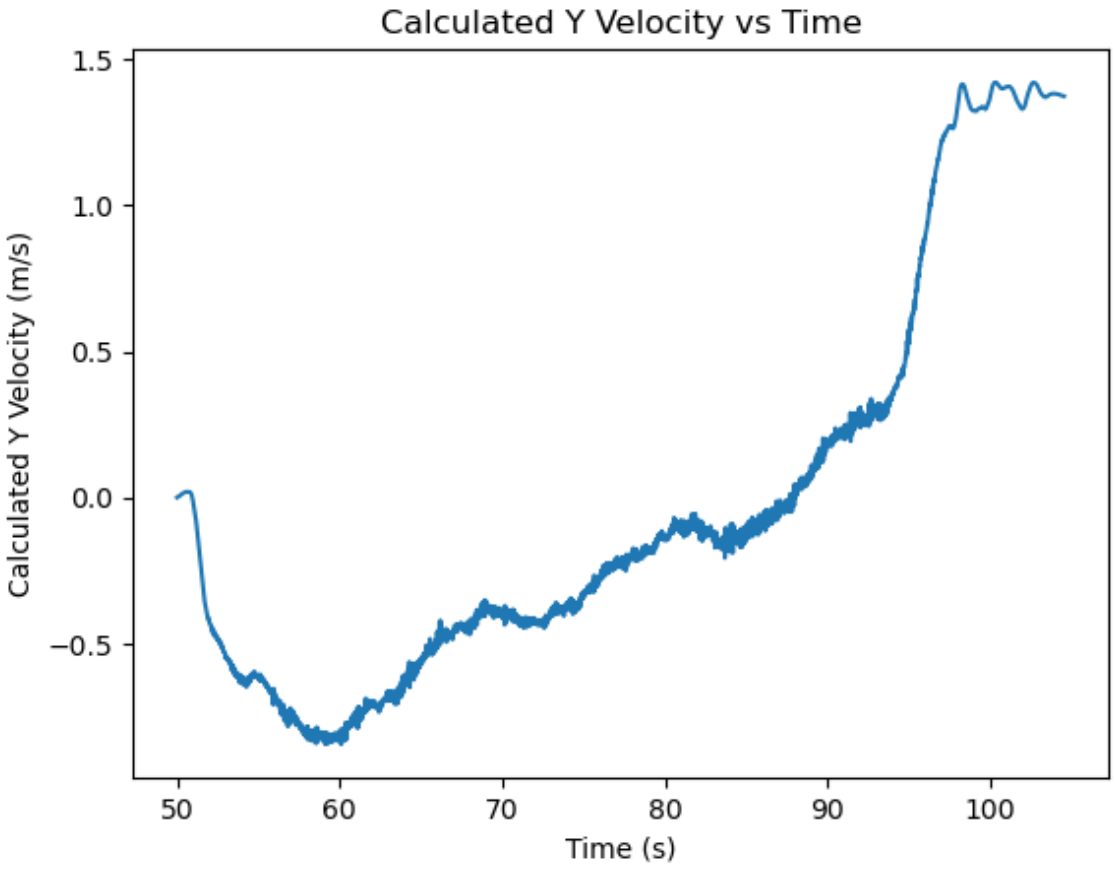
In [9]:
```
P_ay_noise = Pdf_noise["Acceleration y (m/s^2)"]

aynoise = np.mean(P_ay_noise)
P_ay_corr = P_ay_trial2 - aynoise


plt.errorbar(P_t_trial2, P_ay_corr, yerr=P_aerr_trial2, fmt="-")
plt.xlabel("Time (s)")
plt.ylabel("y Acceleration (m/s^2)")
plt.title("y Data Corrected for Noise - Trial 2 (Phone)")
```

Out[9]: Text(0.5, 1.0, 'y Data Corrected for Noise - Trial 2 (Phone)')



In [10]:
```
#I love copy and paste
#FOR Y ACCELERATION

yvelocity=[]
for i in range(P_t_trial2.size):
    yvelocity.append(np.trapz(P_ay_corr [0:i], P_t_trial2[0:i]))

plt.plot(P_t_trial2, yvelocity)
plt.xlabel("Time (s)")
plt.ylabel("Calculated Y Velocity (m/s)")
plt.title("Calculated Y Velocity vs Time")
```

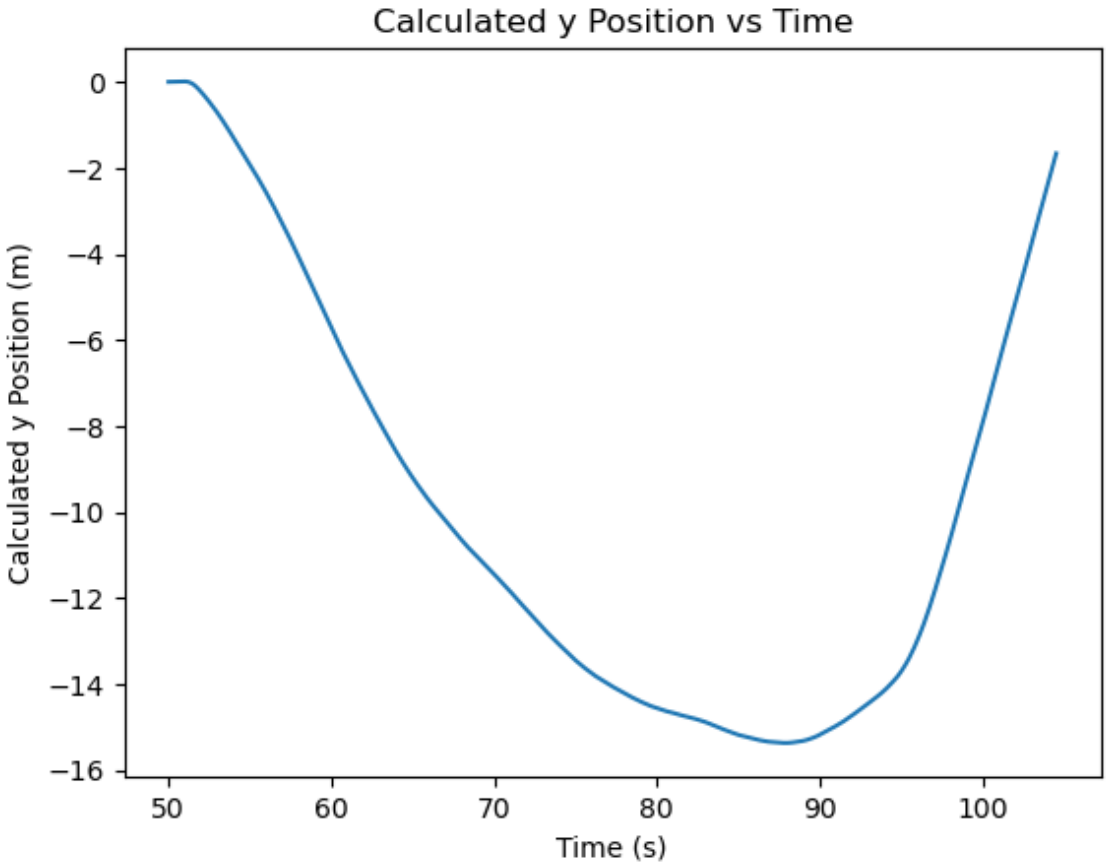
Out[10]:
```
Text(0.5, 1.0, 'Calculated Y Velocity vs Time')
```



This looks a little better - more of the shape we expect, less drift - still drift though - 1.5 m/s over 50 s

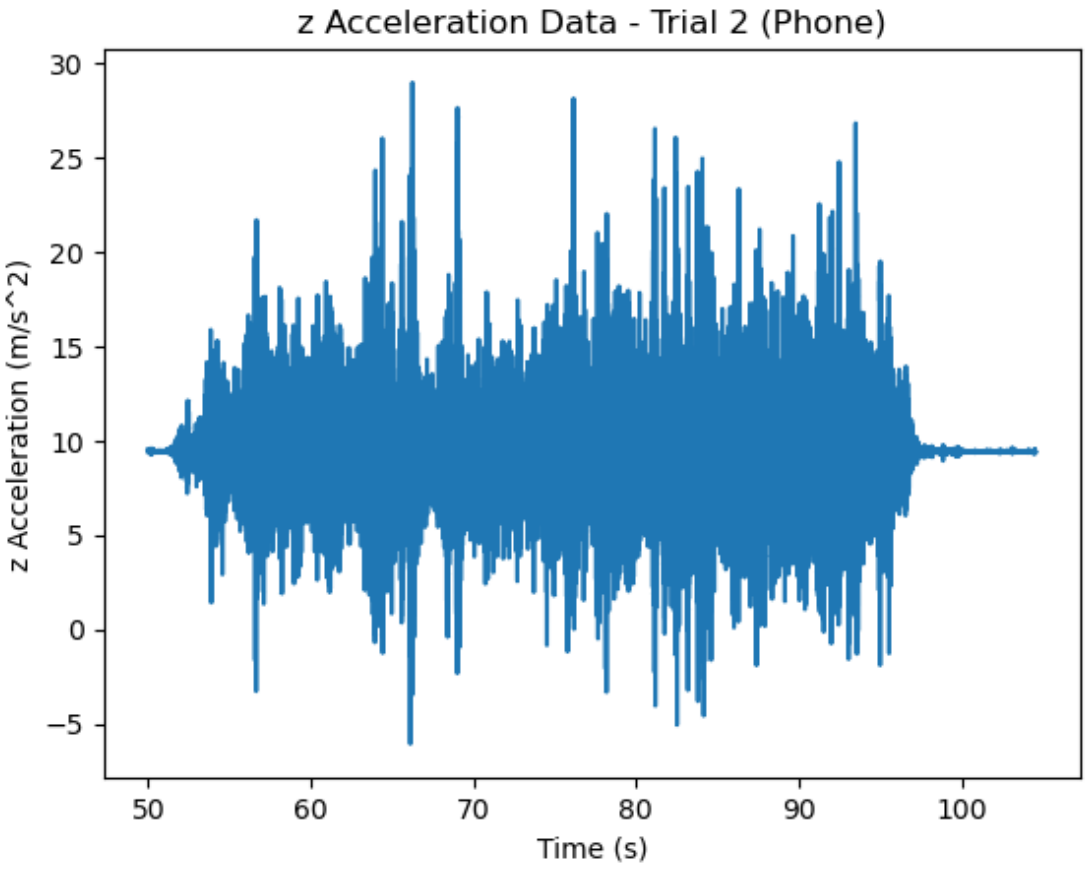
In [11]:
```
yposition=[]
for i in range(P_t_trial2.size):
    yposition.append(np.trapz(yvelocity [0:i], P_t_trial2[0:i]))

plt.plot(P_t_trial2, yposition)
plt.xlabel("Time (s)")
plt.ylabel("Calculated y Position (m)")
plt.title("Calculated y Position vs Time")
```

Out[11]:
```
Text(0.5, 1.0, 'Calculated y Position vs Time')
```



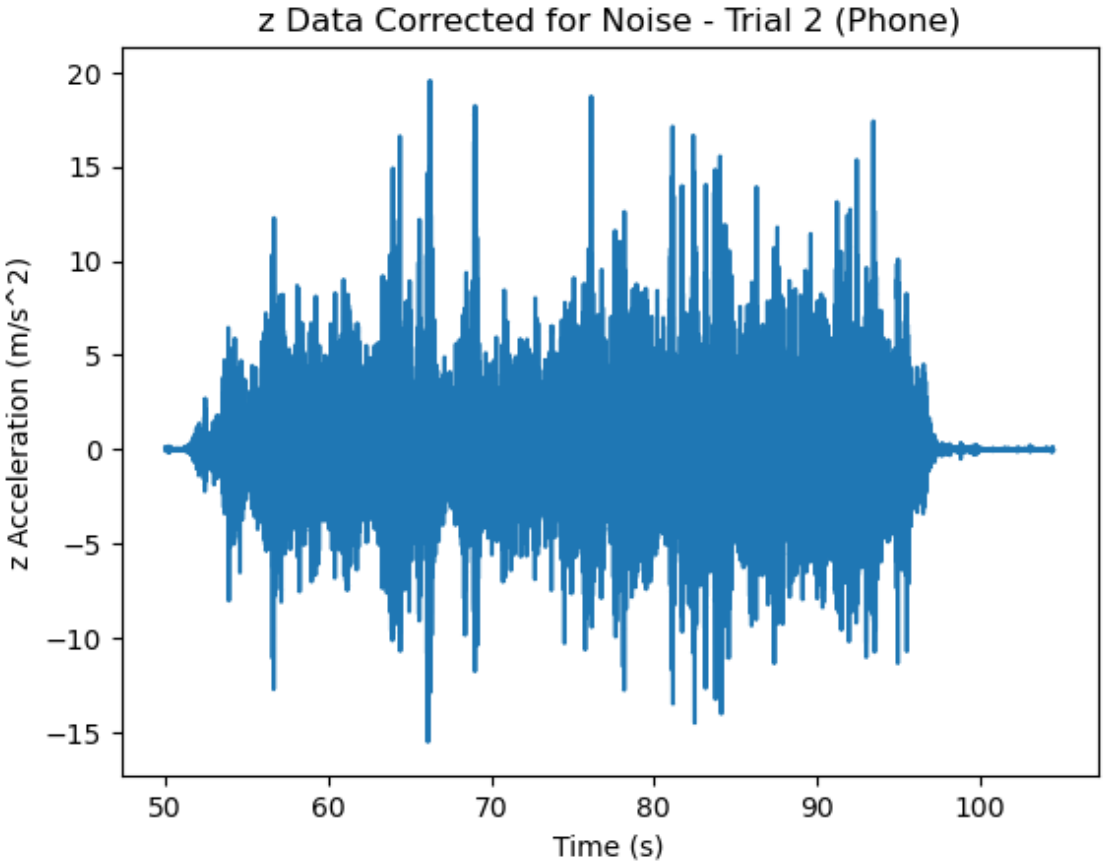
In [12]:
```
P_az_trial2 = P_trial2["Acceleration z (m/s^2)"]

plt.errorbar(P_t_trial2, P_az_trial2, yerr=P_aerr_trial2, fmt="-")
plt.xlabel("Time (s)")
plt.ylabel("z Acceleration (m/s^2)")
plt.title("z Acceleration Data - Trial 2 (Phone)")
```

Out[12]:
```
Text(0.5, 1.0, 'z Acceleration Data - Trial 2 (Phone)')
```



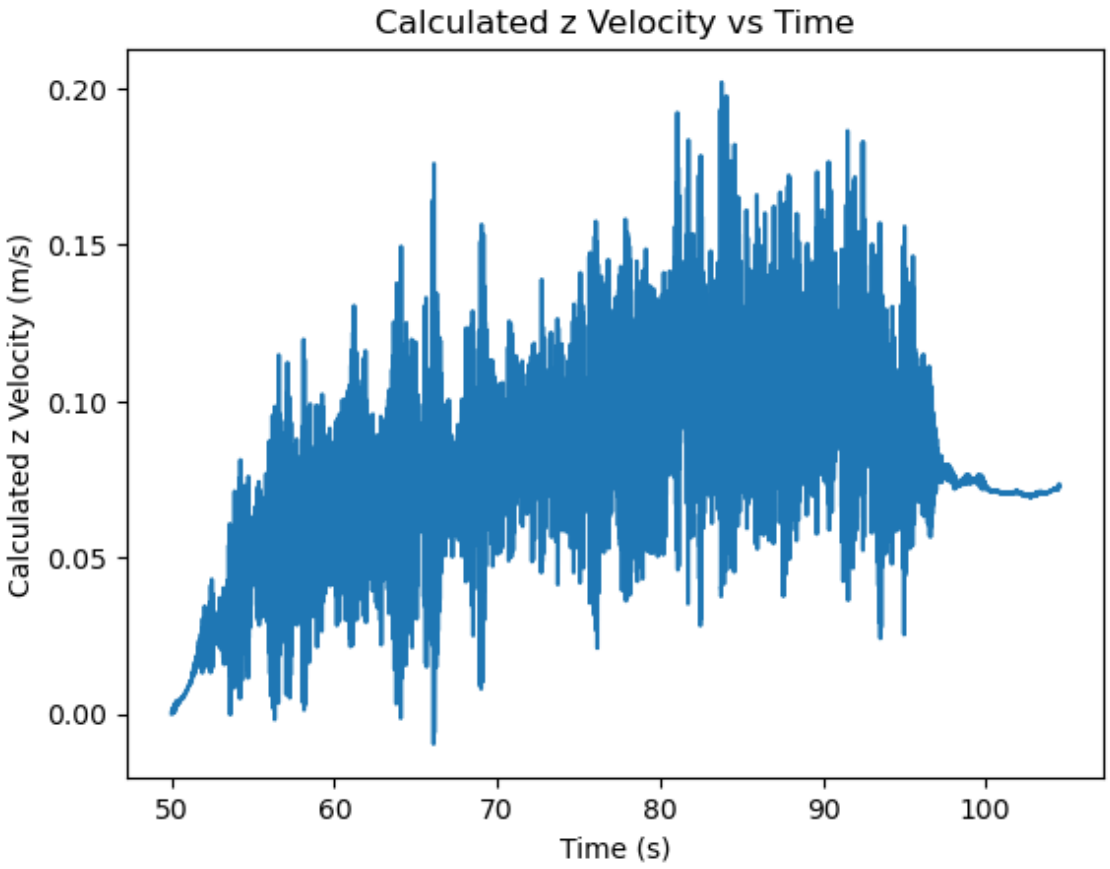
In [13]:
```
P_az_noise = Pdf_noise["Acceleration z (m/s^2)"]

aznoise = np.mean(P_az_noise)
P_az_corr = P_az_trial2 - aznoise


plt.errorbar(P_t_trial2, P_az_corr, yerr=P_aerr_trial2, fmt="-")
plt.xlabel("Time (s)")
plt.ylabel("z Acceleration (m/s^2)")
plt.title("z Data Corrected for Noise - Trial 2 (Phone)")
```

Out[13]: Text(0.5, 1.0, 'z Data Corrected for Noise - Trial 2 (Phone)')



In [14]:
```
#I love copy and paste
#FOR Y ACCELERATION

zvelocity=[]
for i in range(P_t_trial2.size):
    zvelocity.append(np.trapz(P_az_corr [0:i], P_t_trial2[0:i]))

plt.plot(P_t_trial2, zvelocity)
plt.xlabel("Time (s)")
plt.ylabel("Calculated z Velocity (m/s)")
plt.title("Calculated z Velocity vs Time")
```

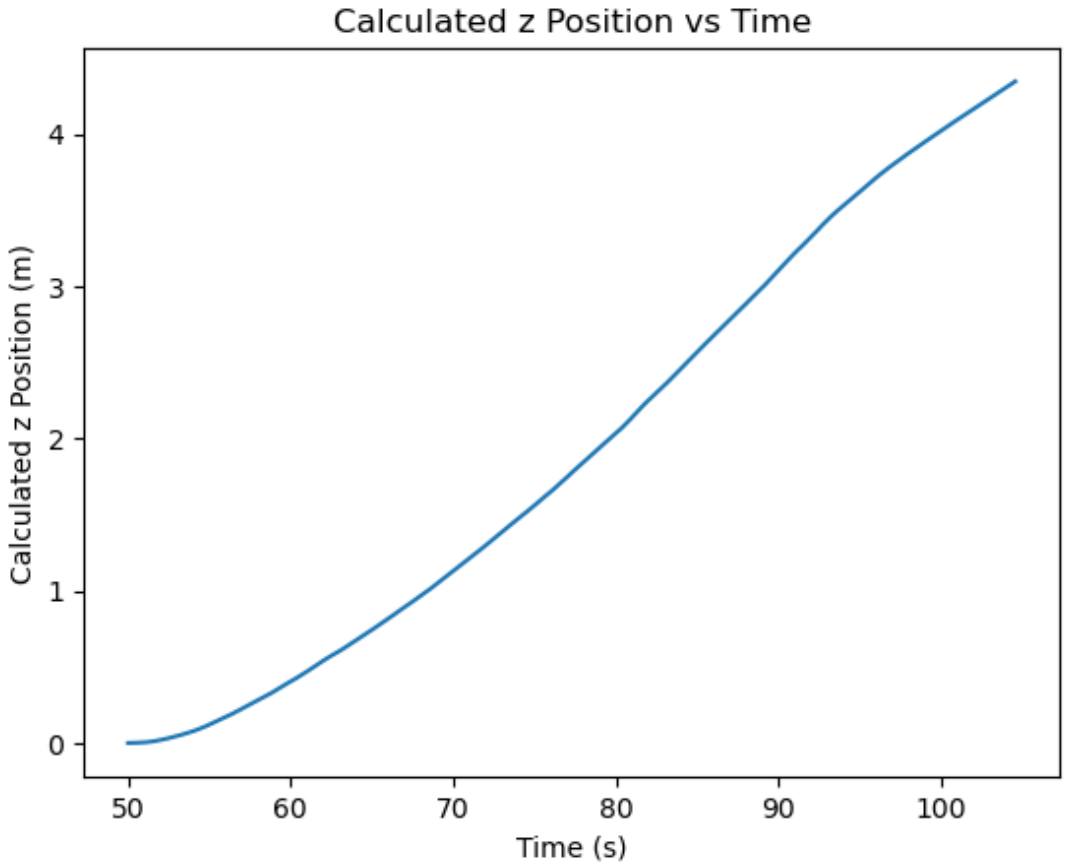
Out[14]:
```
Text(0.5, 1.0, 'Calculated z Velocity vs Time')
```



MORE DRIFT NOTICEABLE HERE - 0.75 over 50 seconds

In [15]:
```
zposition=[]
for i in range(P_t_trial2.size):
    zposition.append(np.trapz(zvelocity [0:i], P_t_trial2[0:i]))

plt.plot(P_t_trial2, zposition)
plt.xlabel("Time (s)")
plt.ylabel("Calculated z Position (m)")
plt.title("Calculated z Position vs Time")
```

Out[15]:
```
Text(0.5, 1.0, 'Calculated z Position vs Time')
```



In [16]:
```
print(xposition[-1],
    yposition[-1],
    zposition[-1])
```

```
-87.41733423885918 -1.6628169122295087 4.343894530201803
```

In [17]:
```
np.sqrt(xposition[-1]**2 +
    yposition[-1]**2 +
    zposition[-1]**2)
```

Out[17]:
```
87.54098871501016
```

In [18]:
```
# ERROR PROPOGATION

pxerr = np.sqrt(3) * P_aerr
pxerr
```

Out[18]:
```
0        0.00866
1        0.00866
2        0.00866
3        0.00866
4        0.00866
          ...
48459    0.00866
48460    0.00866
48461    0.00866
48462    0.00866
48463    0.00866
Name: Acceleration Error (m/s^2), Length: 48464, dtype: float64
```

Compare

87.541 +/- 0.009 m (calculated)

to

52.044 +/- 0.025 m (measured

spoiler: it will not work

In [56]:
```
calc_distance = 87.541
cdistance_err = 0.009
meas_distance = 52.044
mdistance_err = 0.025

d_elev = np.abs(meas_distance - calc_distance)
dd_elev = np.sqrt(mdistance_err**2 + cdistance_err**2)

t_elev = d_elev / dd_elev
print(t_elev)
```

```
1335.9472183641678
```

yikes

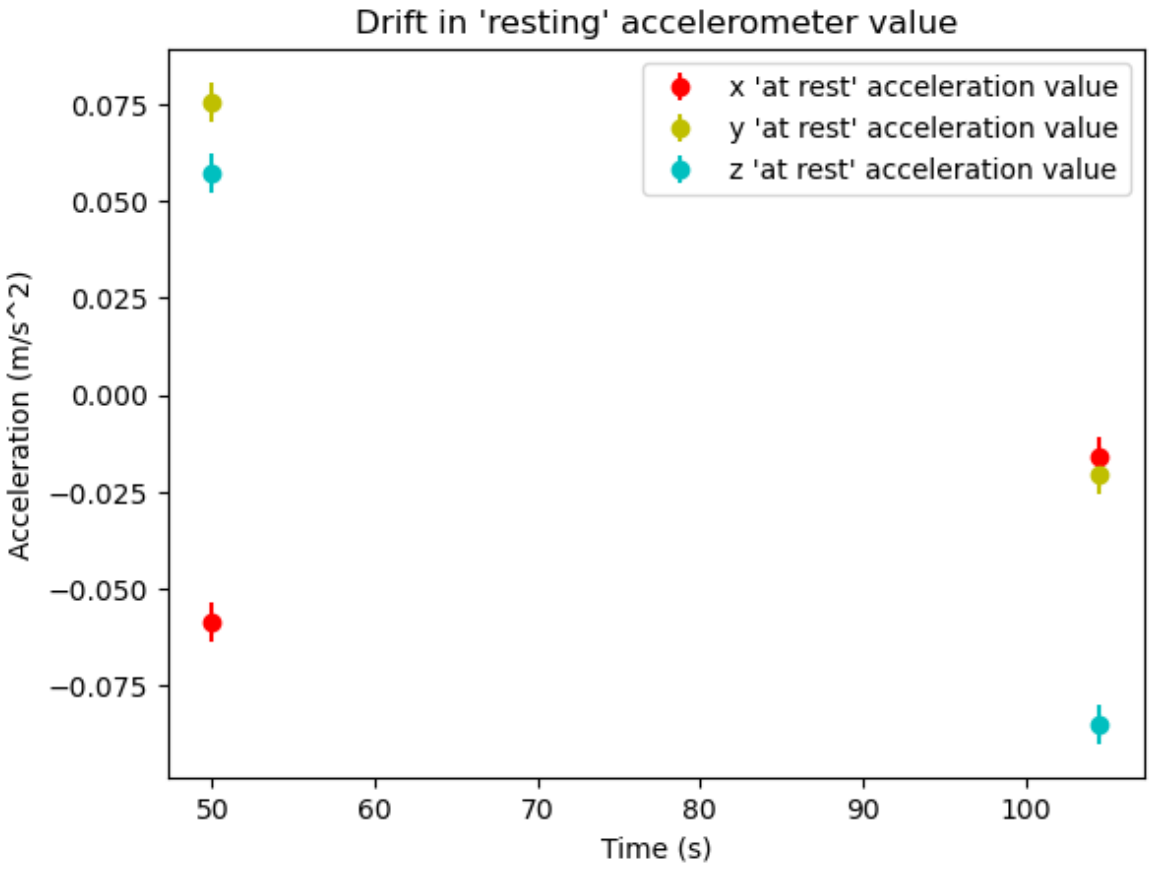
plotting the drift

In [20]:
```
print(P_t_trial2.head(1), P_t_trial2.tail(1))
```

```
23172    50.000891
Name: Time (s), dtype: float64 48463    104.535865
Name: Time (s), dtype: float64
```

In [36]:
```
ti = 50.000891
tf = 104.535865

aerr = P_aerr_trial2.head(1)

PAX_i = P_ax_corr.head(1)
PAX_f = P_ax_corr.tail(1)

PAY_i = P_ay_corr.head(1)
PAY_f = P_ay_corr.tail(1)

PAZ_i = P_az_corr.head(1)
PAZ_f = P_az_corr.tail(1)




plt.errorbar(ti, PAX_i, yerr=aerr, fmt="ro", label="x 'at rest' acceleratio
plt.errorbar(tf, PAX_f, yerr=aerr, fmt="ro")
plt.errorbar(ti, PAY_i, yerr=aerr, fmt="yo", label="y 'at rest' acceleratio
plt.errorbar(tf, PAY_f, yerr=aerr, fmt="yo")
plt.errorbar(ti, PAZ_i, yerr=aerr, fmt="co", label="z 'at rest' acceleratio
plt.errorbar(tf, PAZ_f, yerr=aerr, fmt="co")
plt.xlabel("Time (s)")
plt.ylabel("Acceleration (m/s^2)")
plt.title("Drift in 'resting' accelerometer value")
plt.legend()
```

Out[36]: <matplotlib.legend.Legend at 0x174321190>



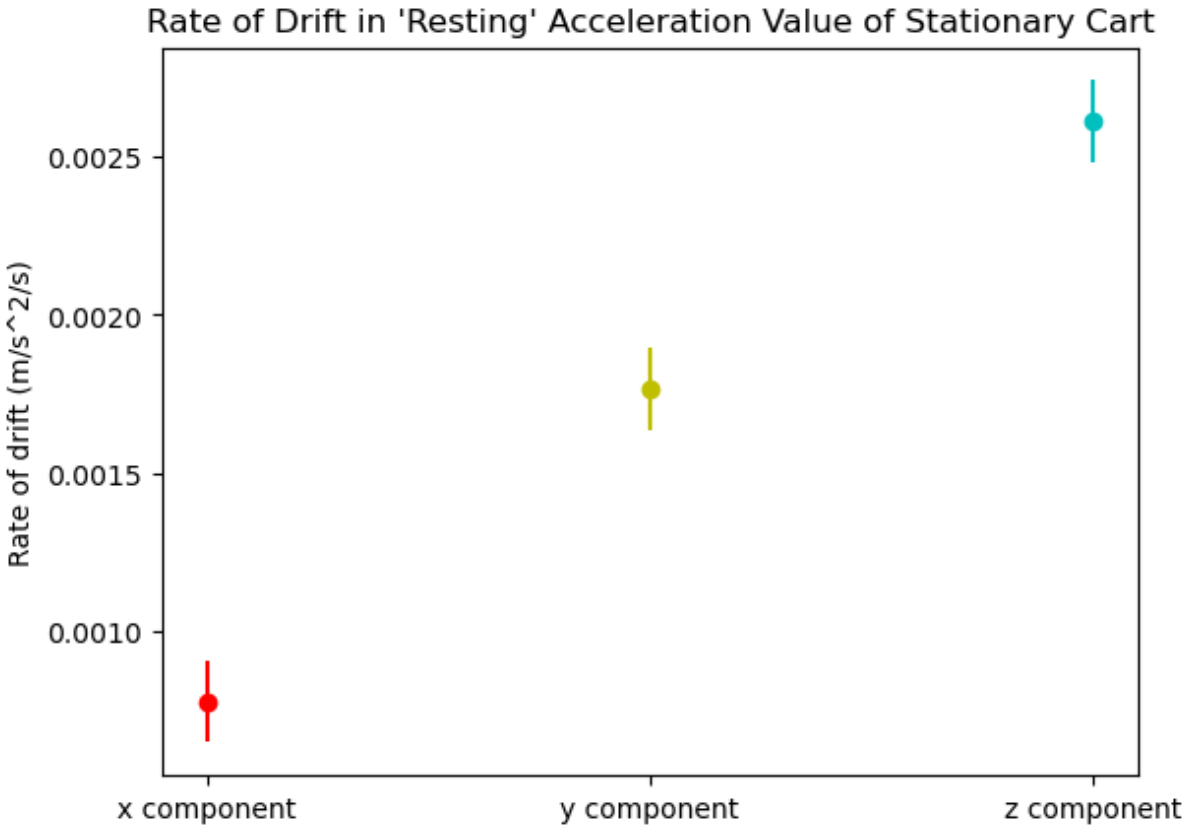
In [48]:
```
xdrift = PAX_f - PAX_i
ydrift = np.abs(PAY_f - PAY_i)
zdrift = np.abs(PAZ_f - PAZ_f)

ddrift = np.sqrt(2) * aerr


print(PAX_i, PAX_f, PAY_i, PAY_f, PAZ_i, PAZ_f, aerr)
```

```
23172   -0.058422
Name: Acceleration x (m/s^2), dtype: float64 48463   -0.015949
Name: Acceleration x (m/s^2), dtype: float64 23172    0.07563
Name: Acceleration y (m/s^2), dtype: float64 48463   -0.020681
Name: Acceleration y (m/s^2), dtype: float64 23172    0.057407
Name: Acceleration z (m/s^2), dtype: float64 48463   -0.084966
Name: Acceleration z (m/s^2), dtype: float64 23172    0.005
Name: Acceleration Error (m/s^2), dtype: float64
```

In [49]:
```
xdrift = np.abs(-0.015949 - (-0.058422))
ydrift = np.abs(-0.020681 - (0.07563))
zdrift = np.abs(-0.084966 - 0.057407)
ddrift = np.sqrt(2) * 0.005
```

In [59]:
```
d_rate_drift = np.sqrt( ((1/(tf-ti))*(ddrift))**2)

plt.errorbar("x component", xdrift/(tf-ti), yerr=d_rate_drift, fmt="ro")
plt.errorbar("y component", ydrift/(tf-ti), yerr=d_rate_drift, fmt="yo")
plt.errorbar("z component", zdrift/(tf-ti), yerr=d_rate_drift, fmt="co")
plt.ylabel("Rate of drift (m/s^2/s)")
plt.title("Rate of Drift in 'Resting' Acceleration Value of Stationary Cart
```

Out[59]:
```
Text(0.5, 1.0, "Rate of Drift in 'Resting' Acceleration Value of Stationa
ry Cart")
```



In [60]:
```
print("xdrift =", xdrift, "+/-", d_rate_drift)
print("ydrift =", ydrift, "+/-", d_rate_drift)
print("zdrift =", zdrift, "+/-", d_rate_drift)
```

```
xdrift = 0.042473 +/- 0.00012966115674439445
ydrift = 0.09631100000000001 +/- 0.00012966115674439445
zdrift = 0.142373 +/- 0.00012966115674439445
```

$drift_x = 0.0425 \pm 0.0001 m/s^2$

$drift_y = 0.0963 \pm 0.0001 m/s^2$

$drift_z = 0.1424 \pm 0.0001 m/s^2$

```
stuff to include: error propogation and contributions

- nasa being silly with the magnetometers
- idk
- error contributions: error from phyphox sensor accuracy
-
```

```
- error contributions: error from phyphox sensor accuracy
-

setup
- sensor mounted to rolling apparatus and pushed along a set path, path
measured with tape measure for comparison
- note:
```